DATABASE MANAGEMENT SYSTEMS · PROJECT REPORT

SafeWay

A time-aware, women-rated location safety database

Domain Spatial & temporal review aggregation

DBMS PostgreSQL 16 with the PostGIS 3.4 extension

Scope Vellore, Tamil Nadu — 75 real locations, 1,130 reviews, 6,780 ratings

Deployment Local (Docker Compose); not a hosted service

Relations 6 base tables, 1 view, 2 enumerated domains

Constraints 6 primary keys, 5 foreign keys, 5 unique, 1 check, 1 trigger

Indexes 11 B-tree, 1 GiST (spatial)

Application Node.js 22 · Express 5 · Prisma 6.19 · React 18

Table of Contents

Part I — Database Design and Implementation

1	Introduction and Problem Statement	1
2	Requirements Analysis	2
3	Conceptual Design — the ER Model	3
4	Logical Design — ER to Relational Mapping	4
5	Normalization	6
6	Physical Design — Data Definition	8
7	Integrity Constraints	10
8	Indexing and Access Paths	11
9	The Derived Score — a View	13
10	Procedural SQL — the Trigger	14
11	Transactions and Concurrency	16
12	Spatial Data with PostGIS	17
13	Query Catalogue	18
14	Performance	21
15	Database Security	22
16	Data Privacy and the Right to Erasure	24
17	Testing the Database	25
18	Schema Evolution	25

Part II — Application and Operation

19	System Architecture	27
20	API Surface	28
21	Interface and Dataset	28
22	Limitations	29
23	Future Work	29
24	Conclusion	29
25	Appendix — Running the System	30
26	References	30

Part I — Database Design and Implementation

1. Introduction and Problem Statement

1.1 Motivation

General-purpose review platforms rate a place with a single score that is an average over its entire population of visitors and over all hours of the day. Both averages destroy information that matters to a woman deciding whether to walk to a particular bus stop alone. A place is not uniformly safe: a bus stand that is crowded, staffed and well lit at two in the afternoon can be deserted and unlit at ten at night. Collapsing those two states into one number is not a rounding error, it is the removal of the only variable the user cares about.

SafeWay is a database-backed system that stores safety assessments along two axes that conventional review schemas discard: *which specific safety factor* is being rated, and *at what time of day* the visit occurred. Aggregate scores are therefore computed per `(location, time_bucket)` pair rather than per location, and the user interface exposes a slider that moves between the four time buckets and recolours the map.

1.2 Why this is a database problem

Almost every interesting property of this system is a data-modelling decision rather than an application feature:

- The central artefact is a **derived aggregate** over a many-to-many relationship (reviews × factors), grouped along a dimension that is itself derived from a timestamp.
- The write rule — only accounts registered as female may insert a review — is a **semantic integrity constraint**, not a UI concern, and is therefore enforced by a trigger inside the database.
- The right-to-erasure requirement conflicts with the need to preserve aggregate data. The resolution is a **referential action** (`ON DELETE SET NULL`) chosen at schema design time, not a deletion routine written later.
- Location proximity is a **spatial query** answered by a GiST-indexed geography column, not by fetching rows and filtering in application code.
- The minimum-sample display rule is enforced in the **view definition**, so no consumer of the data — API, test, or ad-hoc SQL — can bypass it.

1.3 Objectives

#	Objective	Where it is met
O1	Model locations, users, safety factors and time-stamped reviews in a normalised relational schema	§3, §4, §5
O2	Compute safety scores per location *per time bucket*, not per location	§9
O3	Enforce the write rule inside the DBMS so an application bug cannot corrupt data	§10

#	Objective	Where it is met
O4	Answer “places within *r* metres” with a spatial index	§8, §12
O5	Support erasure of a user without destroying aggregate safety data	§16
O6	Guarantee derived attributes cannot be desynchronised by a client	§10.3
O7	Never publish a statistic computed from fewer than three observations	§9.3

2. Requirements Analysis

2.1 Functional requirements

ID	Requirement	Type
FR1	Register an account with an email, password and self-declared gender	Insert
FR2	Authenticate an existing account	Select
FR3	Retrieve all locations within a radius of a point, with scores for all four time buckets	Spatial select
FR4	Retrieve one location with a per-factor breakdown for each time bucket	Aggregate select
FR5	Submit a review: six factor ratings plus a visit timestamp	Multi-table insert
FR6	Permit one review per user per location *per time bucket*	Unique constraint
FR7	Refuse a review from an account not registered as female	Trigger
FR8	Delete an account while preserving its reviews in anonymised form	Update + delete

2.2 Non-functional and integrity requirements

ID	Requirement	Mechanism
NFR1	A rating must lie in the closed interval [1, 5]	`CHECK` constraint
NFR2	A score derived from fewer than 3 reviews must never be published	`CASE` in view
NFR3	The time bucket must always agree with the visit timestamp	`BEFORE INSERT` trigger
NFR4	The write rule must hold even if the application is bypassed	Trigger + test
NFR5	Proximity search must not degrade to a full scan at scale	GiST index
NFR6	A review and its six ratings must commit atomically	Transaction
NFR7	No query may be built by string concatenation of user input	Parameterised SQL
NFR8	Gender must never appear in any unauthenticated response	Projection + test

2.3 Assumptions and scope boundaries

- Gender is **self-declared at registration**. No identity verification is performed. The database enforces the *consequence* of the declaration; establishing its truth is out of scope and discussed in §22.
- All timestamps are stored as `TIMESTAMPTZ` (UTC internally). Time buckets are defined in **Asia/Kolkata**, the single locale the dataset covers.
- The dataset is one city. There is no sharding, partitioning or replication.
- The system runs locally under Docker Compose. There is no production deployment.

3. Conceptual Design — the ER Model

3.1 Entity sets

Entity	Type	Justification
`USER`	Strong	Independently identified by an email address; exists before any review.
`LOCATION_CATEGORY`	Strong	A small controlled vocabulary (bus stop, café, park …). Separated so a category name is stored once.
`LOCATION`	Strong	A physical place with a geographic point. Sourced from OpenStreetMap.
`SAFETY_FACTOR`	Strong	A dimension being rated (lighting, security presence …). Reference data.
`REVIEW`	Strong	One woman's assessment of one place at one time of day. Given a surrogate key because its natural key involves a nullable attribute.
`REVIEW_FACTOR_RATING`	Weak / associative	The score a review gives one factor. Has no identity apart from its parent review; identified by `(review_id, factor_id)`.

3.2 Attributes

Attribute classifications used below: **simple** (atomic), **composite** (decomposable), **derived** (computable from other attributes), **key** (uniquely identifying).

Entity	Attributes
`USER`	id (key, surrogate), *email* (candidate key), password_hash, display_name, gender, created_at
`LOCATION_CATEGORY`	id, *name* (candidate key)
`LOCATION`	id, name, address, **geog** (composite — a point decomposable into latitude and longitude), created_at
`SAFETY_FACTOR`	id, *code* (candidate key), label, display_order
`REVIEW`	id, visited_at, **time_bucket** (*derived* from visited_at), body, created_at
`REVIEW_FACTOR_RATING`	(review_id, factor_id), score

> **Design note — a materialised derived attribute**
>
> `time_bucket` is functionally determined by `visited_at`, so storing it is technically redundant. It is stored deliberately for two reasons. First, the derivation depends on a named time zone, and `visited_at AT TIME ZONE 'Asia/Kolkata'` is `STABLE`, not `IMMUTABLE`; PostgreSQL therefore forbids it inside a `CHECK` constraint, a generated column, or an index expression. Second, the unique

constraint of FR6 must be enforced on the bucket. Materialising it is the only way to place a `UNIQUE` index on it. The redundancy is made safe by having the trigger, not the client, compute the value — see §10.3.

3.3 Relationship sets, cardinality and participation

Relationship	Cardinality	Participation and meaning
`USER` **`writes`** `REVIEW`	1 : N	USER partial (may write none); REVIEW **partial** on the user side — an anonymised review has no author. This is unusual and is the direct consequence of §16.
`CATEGORY` **`classifies`** `LOCATION`	1 : N	LOCATION total — every location has exactly one category.
`LOCATION` **`receives`** `REVIEW`	1 : N	REVIEW total — a review without a location is meaningless.
`REVIEW` **`scores`** `FACTOR`	M : N	Resolved by the associative entity REVIEW_FACTOR_RATING, which carries the descriptive attribute `score`.

3.4 ER diagram

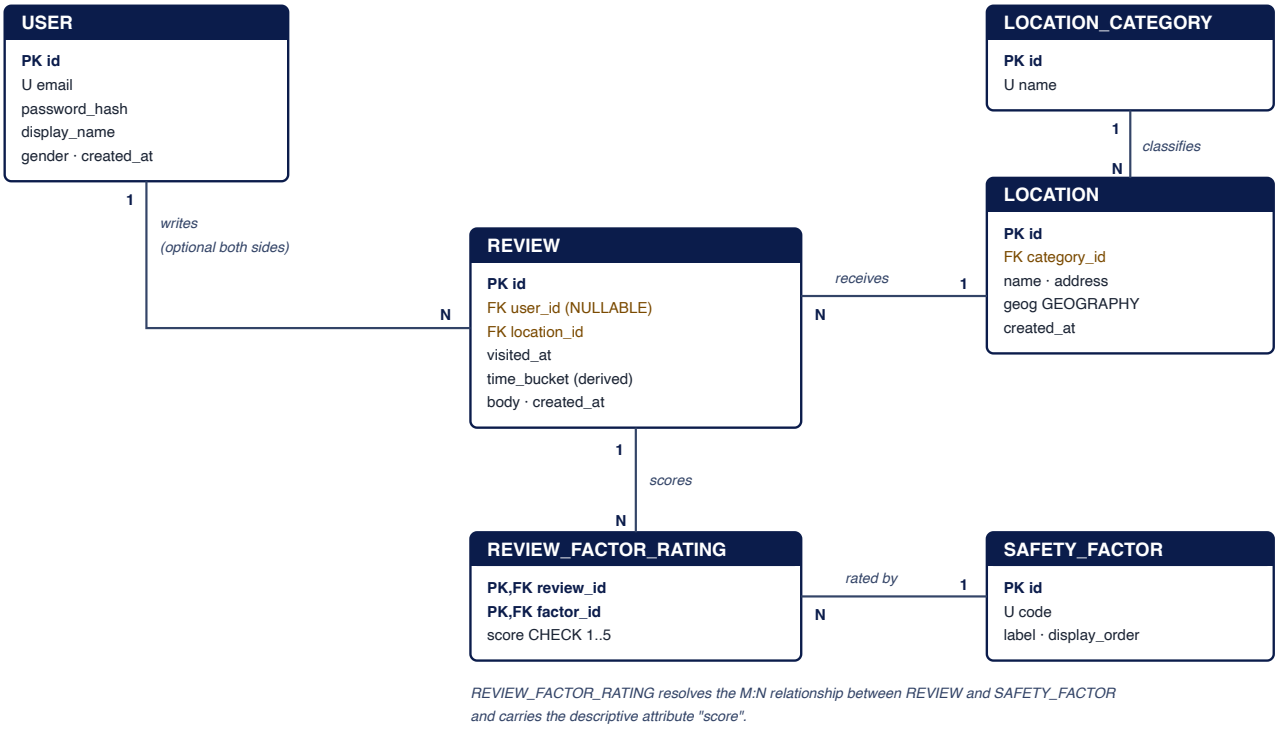


*Figure 1 — Entity-Relationship diagram. **PK** primary key, **FK** foreign key, **U** unique.*

4. Logical Design — ER to Relational Mapping

4.1 Mapping rules applied

Step	Rule	Application in SafeWay
1	Each strong entity becomes a relation; its key attribute becomes the primary key.	`users`, `location_categories`, `locations`, `safety_factors`, `reviews`

Step	Rule	Application in SafeWay
2	Each 1:N relationship is realised by placing the primary key of the "1" side into the "N" side as a foreign key.	`locations.category_id`, `reviews.location_id`, `reviews.user_id`
3	Each M:N relationship becomes its own relation whose primary key is the composite of both participating keys.	`review_factor_ratings(review_id, factor_id)`
4	Descriptive attributes of an M:N relationship live in the resulting relation.	`review_factor_ratings.score`
5	An attribute with a small fixed domain becomes an enumerated type.	`"Gender"`, `"TimeBucket"`
6	Partial participation is expressed by a nullable foreign key.	`reviews.user_id` is `NULL`-able

4.2 Resulting relational schema

```
users(id, email, password_hash, display_name, gender, created_at)
        email UNIQUE

location_categories(id, name)
        name UNIQUE

locations(id, name, category_id, address, geog, created_at)
        category_id  → location_categories(id)

safety_factors(id, code, label, display_order)
        code UNIQUE

reviews(id, user_id, location_id, visited_at, time_bucket, body, created_at)
        user_id      → users(id)        NULLABLE
        location_id  → locations(id)
        UNIQUE(user_id, location_id, time_bucket)

review_factor_ratings(review_id, factor_id, score)
        review_id    → reviews(id)
        factor_id    → safety_factors(id)
        CHECK(score BETWEEN 1 AND 5)

location_safety_score(location_id, time_bucket, score, sample_size)   -- VIEW
```

4.3 Choice of surrogate keys

Every base relation uses an integer surrogate key (`SERIAL`) rather than a natural key. The justification differs per relation and is worth stating explicitly, because surrogate keys are often applied reflexively:

- `users` — `email` is a valid candidate key, but it is mutable and is personal data. A surrogate key means anonymising a user does not require rewriting foreign keys.
- `locations` — no natural key exists. Two distinct bus stops in Vellore genuinely share the name "CMC Hospital"; name is not unique and coordinates are floating-point and therefore unsafe as a key.
- `reviews` — the natural key would be `(user_id, location_id, time_bucket)`, but `user_id` is nullable and SQL forbids `NULL` in a primary key. A surrogate key is *required* here, not merely convenient. The natural key survives as a `UNIQUE` constraint.

- `review_factor_ratings` — deliberately has **no** surrogate key. Its composite key `(review_id, factor_id)` is natural, non-nullable and stable, and doubles as the constraint preventing a review from rating the same factor twice.

5. Normalization

5.1 Functional dependencies

The complete set of non-trivial functional dependencies in the schema:

```
F1   users.id                          → email, password_hash, display_name, gender, created_at
F2   users.email                       → id, password_hash, display_name, gender, created_at

F3   location_categories.id            → name
F4   location_categories.name          → id

F5   locations.id                      → name, category_id, address, geog, created_at

F6   safety_factors.id                 → code, label, display_order
F7   safety_factors.code               → id, label, display_order

F8   reviews.id                        → user_id, location_id, visited_at,
                                         time_bucket, body, created_at
F9   (user_id, location_id, time_bucket) → id, visited_at, body, created_at
F10  reviews.visited_at                → time_bucket        [derivation, see 5.4]

F11  (review_id, factor_id)            → score
```

5.2 Candidate keys

Relation	Candidate keys	Primary key
`users`	`{id}`, `{email}`	`{id}`
`location_categories`	`{id}`, `{name}`	`{id}`
`locations`	`{id}`	`{id}`
`safety_factors`	`{id}`, `{code}`	`{id}`
`reviews`	`{id}`; `{user_id, location_id, time_bucket}` only when `user_id IS NOT NULL`	`{id}`
`review_factor_ratings`	`{review_id, factor_id}`	`{review_id, factor_id}`

5.3 Normal form analysis

First Normal Form (1NF)

1NF requires every attribute to hold a single atomic value from its domain, with no repeating groups. All six relations satisfy 1NF. Two cases merit comment:

- `locations.geog` is a PostGIS `GEOGRAPHY(POINT, 4326)`. It is internally decomposable into latitude and longitude, which looks like a composite attribute. It is treated as atomic because the DBMS defines it as a single-valued domain with its own operators (`ST_DWithin`, `ST_Distance`) and index support.

Splitting it into two `DOUBLE PRECISION` columns would *lose* the ability to index it spatially, which is the opposite of what normalization is for. This is the standard treatment of user-defined types under 1NF.

- The six factor ratings of a review are **not** six columns of `reviews`. That design would be a repeating group and a 1NF violation, and would additionally require a schema change to add a seventh factor. They are rows in `review_factor_ratings` instead.

Second Normal Form (2NF)

2NF requires 1NF plus no partial dependency of a non-prime attribute on part of a composite candidate key. Only one relation has a composite key:

```
review_factor_ratings(review_id, factor_id, score)

   Is  review_id → score  ?   No — one review has six different scores.
   Is  factor_id → score  ?   No — one factor is scored differently by different reviews.
   score is fully functionally dependent on {review_id, factor_id}.    ⇒ 2NF holds.
```

All other relations have a single-attribute primary key, so partial dependency is impossible and 2NF is satisfied vacuously.

Third Normal Form (3NF)

3NF requires 2NF plus no transitive dependency of a non-prime attribute on the primary key. The critical case is `locations`, and it is the reason the category was extracted into its own relation:

```
Rejected design:
   locations(id, name, category_name, address, geog)
      id → category_name        and         category_name is non-prime
   Storing the literal 'Bus stop' on all 16 bus-stop rows creates update anomalies:
      · UPDATE anomaly — renaming the category requires touching every row.
      · INSERT anomaly — a category cannot exist before a location uses it.
      · DELETE anomaly — deleting the last bus stop erases the category itself.

Adopted design:
   location_categories(id, name)
   locations(id, name, category_id, address, geog)
      id → category_id          and         category_id is a foreign key, not a fact
   All three anomalies are removed.                                    ⇒ 3NF holds.
```

The same reasoning produced `safety_factors`: the label “Free from harassment” is stored exactly once, not on each of the 6,780 rating rows.

Boyce-Codd Normal Form (BCNF)

BCNF requires that for every non-trivial dependency $X \rightarrow Y$, X is a superkey.

Relation	Determinants	BCNF
`users`	`{id}`, `{email}` — both candidate keys	Yes
`location_categories`	`{id}`, `{name}` — both candidate keys	Yes
`locations`	`{id}` only	Yes
`safety_factors`	`{id}`, `{code}` — both candidate keys	Yes
`reviews`	`{id}`, the natural key, and `{visited_at}` — see below	Qualified

Relation	Determinants	BCNF
`review_factor_ratings`	`{review_id, factor_id}` only	Yes

5.4 The one deliberate deviation

Dependency **F10**, `visited_at → time_bucket`, is a genuine functional dependency whose determinant `{visited_at}` is *not* a superkey of `reviews` — two different women can visit the same place at the same instant. Strictly, `reviews` is therefore not in BCNF.

The textbook remedy is to decompose into `reviews(id, …, visited_at)` and `bucket_of(visited_at, time_bucket)`. This is rejected, and the reasoning is the substance of the trade-off:

- The decomposed relation would hold one row per distinct timestamp — effectively as many rows as reviews — while eliminating no redundancy of consequence.
- Every read of the score view would require an additional join purely to recover a value that is a pure function of a column already present.
- The unique constraint of FR6 must range over `time_bucket`. After decomposition it would span two relations and could no longer be expressed as a single `UNIQUE` index.

The anomaly BCNF protects against is an inconsistent pair — a row whose `time_bucket` disagrees with its `visited_at`. That anomaly is instead eliminated *procedurally*: the `BEFORE INSERT` trigger of §10 overwrites `time_bucket` on every insert, so no client can write an inconsistent pair even by trying. This is a conscious exchange of a declarative guarantee for a procedural one, made because the declarative form is unavailable (the derivation is not `IMMUTABLE` and so cannot be a generated column or a `CHECK`).

> **Summary**
>
> Five of six relations are in **BCNF**. `reviews` is in **3NF** and violates BCNF on exactly one derivation dependency, which is neutralised by a trigger. No relation exhibits an update, insert or delete anomaly. Multivalued dependencies do not arise, so 4NF is satisfied trivially.

6. Physical Design — Data Definition

6.1 Enumerated domains

```
CREATE TYPE "Gender"     AS ENUM ('female', 'male', 'other');
CREATE TYPE "TimeBucket" AS ENUM ('morning', 'afternoon', 'evening', 'night');
```

An `ENUM` is preferred to a `VARCHAR` + `CHECK` because it enforces the domain at the type level, sorts in declaration order rather than alphabetically (so `morning < afternoon < evening < night` orders correctly), and stores as a 4-byte OID rather than a string.

6.2 Table definitions

```
CREATE EXTENSION IF NOT EXISTS postgis;

CREATE TABLE users (
```

```
    id            SERIAL       PRIMARY KEY,
    email         TEXT         NOT NULL UNIQUE,
    password_hash TEXT         NOT NULL,
    display_name  TEXT         NOT NULL,
    gender        "Gender"     NOT NULL,
    created_at    TIMESTAMPTZ(6) NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE location_categories (
    id   SERIAL PRIMARY KEY,
    name TEXT   NOT NULL UNIQUE
);

CREATE TABLE locations (
    id          SERIAL PRIMARY KEY,
    name        TEXT   NOT NULL,
    category_id INTEGER NOT NULL
                REFERENCES location_categories(id) ON UPDATE CASCADE ON DELETE RESTRICT,
    address     TEXT,
    geog        GEOGRAPHY(POINT, 4326) NOT NULL,
    created_at  TIMESTAMPTZ(6) NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX locations_geog_idx ON locations USING GIST (geog);

CREATE TABLE safety_factors (
    id            SERIAL PRIMARY KEY,
    code          TEXT   NOT NULL UNIQUE,
    label         TEXT   NOT NULL,
    display_order INTEGER NOT NULL
);

CREATE TABLE reviews (
    id          SERIAL PRIMARY KEY,
    user_id     INTEGER      REFERENCES users(id)     ON UPDATE CASCADE ON DELETE SET NULL,
    location_id INTEGER NOT NULL
                             REFERENCES locations(id) ON UPDATE CASCADE ON DELETE CASCADE,
    visited_at  TIMESTAMPTZ(6) NOT NULL,
    time_bucket "TimeBucket"   NOT NULL,
    body        TEXT,
    created_at  TIMESTAMPTZ(6) NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE UNIQUE INDEX reviews_user_id_location_id_time_bucket_key
    ON reviews (user_id, location_id, time_bucket);
CREATE INDEX reviews_location_id_time_bucket_idx
    ON reviews (location_id, time_bucket);

CREATE TABLE review_factor_ratings (
    review_id INTEGER NOT NULL
              REFERENCES reviews(id)        ON UPDATE CASCADE ON DELETE CASCADE,
    factor_id INTEGER NOT NULL
              REFERENCES safety_factors(id) ON UPDATE CASCADE ON DELETE RESTRICT,
    score     SMALLINT NOT NULL,
    PRIMARY KEY (review_id, factor_id),
    CONSTRAINT review_factor_ratings_score_check CHECK (score BETWEEN 1 AND 5)
);
```

Listing 1 — Data definition language, reproduced from the live database via `information_schema` and `pg_constraint`.

6.3 Data types and rationale

Type	Storage	Why chosen
`SERIAL / INTEGER`	4 bytes	Sufficient for 2.1×10^9 rows. `BIGINT` rejected because JavaScript cannot represent a 64-bit integer natively, so it would serialise incorrectly in the API layer.
`SMALLINT`	2 bytes	A rating is 1–5. Used on the largest table (6,780 rows) where width matters most.
`TEXT`	variable	In PostgreSQL `TEXT` and `VARCHAR(n)` have identical performance; an arbitrary length cap adds no value and causes migrations later.
`TIMESTAMPTZ`	8 bytes	Stores an absolute instant. `TIMESTAMP` without zone would make the Asia/Kolkata bucket derivation ambiguous.
`GEOGRAPHY(POINT,4326)`	variable	Distances computed on the WGS-84 spheroid in metres. `GEOMETRY` would return degrees, requiring a projection step.
`NUMERIC (view)`	variable	The score is produced by `ROUND(AVG(…),2)` ; exact decimal avoids binary floating-point display artefacts.

7. Integrity Constraints

7.1 Classification

Category	Mechanism	Instances in SafeWay
Domain	Data type, `ENUM` , `NOT NULL` , `CHECK`	19 `NOT NULL` columns; 2 enumerated domains; `score BETWEEN 1 AND 5`
Entity	`PRIMARY KEY`	6 — five surrogate, one composite
Key	`UNIQUE`	4 single-column (`users.email` , `location_categories.name` , `safety_factors.code`) plus the composite `(user_id, location_id, time_bucket)`
Referential	`FOREIGN KEY`	5, each with an explicit `ON DELETE` action
Semantic	Trigger	The write rule and the bucket derivation (§10)
Statistical	View predicate	Minimum three samples before a score is published (§9.3)

7.2 Referential actions and why each was chosen

The `ON DELETE` action is the single most consequential choice in this schema. Each of the five foreign keys uses a different rule, and the differences are deliberate.

Foreign key	On delete	Reason
`reviews.user_id → users`	**SET NULL**	A woman may erase her account. Her reviews must survive as anonymous rows so the aggregate safety data other women depend on is not silently degraded. Requires the column to be nullable — see §16.
`reviews.location_id → locations`	**CASCADE**	A review of a place that no longer exists is meaningless. Deleting the place should remove its reviews.

Foreign key	On delete	Reason
`review_factor_ratings.review_id → reviews`	**CASCADE**	Ratings are existence-dependent on their parent review; orphans would corrupt every aggregate.
`review_factor_ratings.factor_id → safety_factors`	**RESTRICT**	Reference data. Deleting "lighting" while 1,130 reviews rate it would destroy history; the deletion is refused instead.
`locations.category_id → location_categories`	**RESTRICT**	Same reasoning. A category in use cannot be removed.

Observation

The default `ON DELETE` action in SQL is `NO ACTION`. Leaving all five at the default would have made account deletion impossible without first destroying the user's reviews — converting a privacy feature into data loss. The actions above were chosen during schema design precisely because retrofitting them later requires rewriting rows, not just constraints.

7.3 The NULL semantics that make FR6 work

`UNIQUE(user_id, location_id, time_bucket)` interacts with SQL's three-valued logic in a way that is useful here. Under the SQL standard, two rows whose `user_id` is `NULL` are *not* considered duplicates, because `NULL = NULL` evaluates to `UNKNOWN` rather than `TRUE`. Consequently:

- A live user is limited to exactly four reviews per location, one per bucket.
- Anonymised reviews (`user_id IS NULL`) are exempt from the constraint and can accumulate without collision.

Had the constraint been declared `UNIQUE NULLS NOT DISTINCT` (available from PostgreSQL 15), the second anonymised review of any location would have been rejected, breaking erasure. The default behaviour is the correct one and is relied upon deliberately.

8. Indexing and Access Paths

8.1 Index inventory

Index	Type	Origin	Purpose
`users_pkey`	B-tree	PK	Row lookup by id
`users_email_key`	B-tree	UNIQUE	Login lookup; enforces the candidate key
`location_categories_pkey`	B-tree	PK	Join target from locations
`location_categories_name_key`	B-tree	UNIQUE	Candidate key
`locations_pkey`	B-tree	PK	Detail lookup
`locations_geog_idx`	**GiST**	Explicit	**Spatial proximity search**
`safety_factors_pkey / _code_key`	B-tree	PK / UNIQUE	Reference lookup
`reviews_pkey`	B-tree	PK	Row lookup

Index	Type	Origin	Purpose
reviews_user_id_location_id_time_bucket_key	B-tree	UNIQUE	**Enforces FR6**
reviews_location_id_time_bucket_idx	B-tree	Explicit	Composite index supporting the grouping in the score view
review_factor_ratings_pkey	B-tree	PK	Composite key; also the join path from reviews

8.2 Why GiST and not B-tree for `geog`

A B-tree imposes a total order on its keys. Geographic points have no meaningful total order: sorting by latitude puts two points 1 km apart on the same parallel arbitrarily far apart in the index. A **Generalized Search Tree (GiST)** instead stores a hierarchy of bounding boxes, so the question "which points fall inside this box?" is answered by descending only the subtrees whose bounding box intersects the query box. This is what makes `ST_DWithin` sub-linear.

8.3 Measured plan — an honest result

Forcing the planner to use the spatial index confirms it is valid and usable:

```
SET enable_seqscan = off;
EXPLAIN (ANALYZE, COSTS OFF)
SELECT l.id FROM locations l
WHERE ST_DWithin(l.geog, ST_SetSRID(ST_MakePoint(79.1325,12.9165),4326)::geography, 12000);

 Index Scan using locations_geog_idx on locations l  (actual rows=75 loops=1)
   Index Cond: (geog && _st_expand('...'::geography, 12000::double precision))
   Filter: st_dwithin(geog, '...'::geography, 12000::double precision, true)
 Execution Time: 4.380 ms
```

With the planner left alone, however, it chooses a sequential scan:

```
 Seq Scan on locations l  (actual rows=75 loops=1)
   Filter: st_dwithin(geog, '...'::geography, 12000::double precision, true)
   Buffers: shared hit=107
 Execution Time: 7.226 ms
```

This is **correct planner behaviour, not a defect**. The table holds 75 rows on 107 pages and the predicate selects all 75 of them. Reading the whole heap is cheaper than descending an index and then reading the whole heap anyway. The index earns its cost only when selectivity is high — that is, at thousands of locations, or with a small radius. Reporting this honestly is more useful than presenting the forced plan as though it were the natural one.

8.4 The `&&` operator — two-phase spatial filtering

Note that the index condition in the forced plan is `geog && _st_expand(...)`, not `ST_DWithin` itself. PostGIS evaluates spatial predicates in two phases:

1. **Index (cheap) filter** — `&&` tests bounding-box overlap using the GiST index. Fast but approximate: it admits false positives near box corners.

2. **Exact (expensive) filter** — `ST_DWithin` computes true spheroidal distance on the surviving candidates only.

The combination gives exact results while restricting the expensive geodesic computation to a small candidate set.

9. The Derived Score — a View

9.1 Definition

```
CREATE VIEW location_safety_score AS
WITH review_means AS (
    SELECT r.id            AS review_id,
           r.location_id,
           r.time_bucket,
           AVG(rfr.score)::numeric AS review_mean
    FROM   reviews r
    JOIN   review_factor_ratings rfr ON rfr.review_id = r.id
    GROUP  BY r.id, r.location_id, r.time_bucket
)
SELECT location_id,
       time_bucket,
       CASE WHEN COUNT(*) >= 3
            THEN ROUND(AVG(review_mean), 2)
            ELSE NULL
       END           AS score,
       COUNT(*)::int AS sample_size
FROM   review_means
GROUP  BY location_id, time_bucket;
```

9.2 A two-stage aggregation, and why it is not one

The computation is deliberately a mean of means rather than a single flat average:

```
Stage 1 (CTE)   per review    :  review_mean = mean of that review's 6 factor scores
Stage 2 (outer) per (loc,bkt) :  score       = mean of the review_means in that group
```

Flattening it to `AVG(rfr.score)` over the join would weight each *rating* equally instead of each *review*. Those differ whenever reviews do not all carry the same number of ratings — which the schema permits, since nothing forces a review to rate all six factors. The two-stage form guarantees one review contributes exactly one observation regardless, which is the semantics the product requires.

9.3 The minimum-sample rule as a schema-level guarantee

The `CASE WHEN COUNT(*) >= 3` clause is the most important line in the view. Placing it in the view rather than in the API means *no* consumer can bypass it: not the application, not a test, not a student running ad-hoc SQL. Verified on the live database:

```
SELECT count(*) AS violations
FROM   location_safety_score
WHERE  sample_size < 3 AND score IS NOT NULL;
```

```
 violations
------------
          0
```

9.4 View versus materialised view versus table

Option	Consequence	Verdict
Base table + trigger	Requires an `AFTER INSERT` trigger to recompute and upsert the affected row. Introduces a cache that can drift from its source, and a second write path to keep correct.	Rejected
Materialised view	Fast reads, but stale until `REFRESH`. A review would not affect the map until a refresh ran.	Rejected at this scale
Plain view	Always consistent by construction; zero invalidation logic; the score changes the instant the underlying rows change. Cost is recomputation per query — measured at 37 ms over 6,780 ratings.	**Adopted**

The migration path is stated for completeness: at a scale where 37 ms becomes unacceptable, the same SQL becomes a `MATERIALIZED VIEW` with a refresh triggered on write. Because consumers reference a relation name and not the query, that change is invisible to the application.

10. Procedural SQL — the Trigger

10.1 Purpose

A single `BEFORE INSERT` trigger on `reviews` performs three jobs that cannot be expressed declaratively: it validates the author against a rule that depends on another table, it rejects authorless inserts, and it computes a derived column.

10.2 Source

```
CREATE OR REPLACE FUNCTION safeway_review_gate() RETURNS trigger AS $$
DECLARE
  author_gender "Gender";
  ist_hour      int;
BEGIN
  IF NEW.user_id IS NULL THEN
    RAISE EXCEPTION 'a review must have an author'
      USING ERRCODE = 'check_violation';                    -- SQLSTATE 23514
  END IF;

  SELECT gender INTO author_gender FROM users WHERE id = NEW.user_id;

  IF author_gender IS NULL THEN
    RAISE EXCEPTION 'unknown author %', NEW.user_id
      USING ERRCODE = 'foreign_key_violation';               -- SQLSTATE 23503
  END IF;

  IF author_gender <> 'female' THEN
    RAISE EXCEPTION 'only accounts registered as female may post reviews'
      USING ERRCODE = 'insufficient_privilege';              -- SQLSTATE 42501
  END IF;

  -- morning 05–11 · afternoon 11–17 · evening 17–21 · night 21–05, in IST.
```

```
  ist_hour := EXTRACT(HOUR FROM (NEW.visited_at AT TIME ZONE 'Asia/Kolkata'));
  NEW.time_bucket := (CASE
    WHEN ist_hour >= 5  AND ist_hour < 11 THEN 'morning'
    WHEN ist_hour >= 11 AND ist_hour < 17 THEN 'afternoon'
    WHEN ist_hour >= 17 AND ist_hour < 21 THEN 'evening'
    ELSE 'night'
  END)::"TimeBucket";

  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER reviews_gate_and_derive
  BEFORE INSERT ON reviews
  FOR EACH ROW EXECUTE FUNCTION safeway_review_gate();
```

Listing 2 — retrieved with `pg_get_functiondef()` from the live database.

10.3 Why `BEFORE` and not `AFTER`

The timing is load-bearing, not incidental. Only a `BEFORE` trigger may modify `NEW` and have the modification persist; an `AFTER` trigger sees the row already written and its assignment to `NEW` is discarded. Because the trigger assigns `NEW.time_bucket`, it must be `BEFORE`.

The ordering has a second, subtler consequence. PostgreSQL evaluates constraints in this order:

```
INSERT statement
     ↓
BEFORE ROW trigger        ← safeway_review_gate() runs here; NEW.time_bucket is overwritten
     ↓
NOT NULL / CHECK / domain checks
     ↓
UNIQUE and PRIMARY KEY index checks   ← sees the DERIVED bucket, not the client's value
     ↓
FOREIGN KEY checks
     ↓
row is written
```

The unique index of FR6 is therefore evaluated against the bucket the *database* computed. A client cannot evade the one-review-per-bucket rule by sending a `time_bucket` that disagrees with its `visited_at`; the value it sends is overwritten before the index is consulted. This is the mechanism referred to in §5.4.

10.4 Verification of the derivation

All eight bucket boundaries were exercised by inserting directly through a raw `libpq` connection and reading back the stored value:

visited_at (IST)	Derived bucket	Boundary
2026-09-01 04:59	night	upper edge of night
2026-09-01 05:00	morning	lower edge of morning
2026-09-01 10:59	morning	upper edge of morning
2026-09-01 11:00	afternoon	lower edge of afternoon
2026-09-01 16:59	afternoon	upper edge

visited_at (IST)	Derived bucket	Boundary
2026-09-01 17:00	evening	lower edge of evening
2026-09-01 20:59	evening	upper edge
2026-09-01 21:00	night	lower edge of night

Independent cross-check

The seed program computes the same bucket independently in application code before inserting. Re-seeding with the trigger active produced a byte-identical MD5 digest of the entire score view (`74263475f381d0ae45906ed691d49981`) compared with the pre-trigger run across all 1,130 reviews. Two independent implementations of the same rule agreeing on every row is stronger evidence than either passing its own test.

11. Transactions and Concurrency

11.1 The one multi-statement write

Submitting a review writes to two tables and must be all-or-nothing: a review row with no ratings would appear in `COUNT(*)` while contributing no score, corrupting every aggregate for that location and bucket.

```
BEGIN;
    INSERT INTO reviews (user_id, location_id, visited_at, time_bucket, body)
    VALUES ($1, $2, $3, 'morning', $4)          -- placeholder; the trigger overwrites it
    RETURNING id, time_bucket;

    INSERT INTO review_factor_ratings (review_id, factor_id, score)
    VALUES ($5,$6,$7), ($5,$8,$9), ($5,$10,$11),
           ($5,$12,$13), ($5,$14,$15), ($5,$16,$17);   -- one statement, six rows
COMMIT;
```

The six ratings are written as a **single multi-row** `INSERT` rather than six statements. This reduces round trips, and makes the write atomic at statement level as well as transaction level.

11.2 ACID properties in this schema

Property	How it is realised here
Atomicity	The review and its six ratings commit together or not at all. If the trigger raises `42501` mid-transaction, the already-inserted review row is rolled back.
Consistency	Every constraint in §7 holds at commit. The trigger additionally guarantees the derived-attribute invariant that no declarative constraint could express.
Isolation	PostgreSQL default `READ COMMITTED` , implemented by MVCC. A concurrent reader of the score view sees a consistent snapshot and never a review with a partial set of ratings.
Durability	Write-ahead logging. A committed review survives process termination; the WAL is replayed on restart.

11.3 Isolation level analysis

`READ COMMITTED` is sufficient and no stronger level is required. The reasoning:

- The only multi-statement transaction writes two tables and reads neither, so it cannot observe a non-repeatable read.
- The score view is read in a single statement, which under MVCC sees one snapshot. A phantom review inserted concurrently is simply not in that snapshot; it appears on the next read. For an advisory safety score this is acceptable — the value is a statistical estimate, not an account balance.
- The uniqueness of FR6 is enforced by an index, not by a read-then-write pattern, so it is immune to the lost-update anomaly that would otherwise require `SERIALIZABLE`. Two concurrent inserts of the same `(user, location, bucket)` triple cannot both succeed: one receives `SQLSTATE 23505` regardless of interleaving.

11.4 Error codes surfaced to the application

SQLSTATE	Condition	HTTP	Raised by
42501	insufficient_privilege	403	Trigger — non-female author
23505	unique_violation	409	FR6 unique index
23503	foreign_key_violation	404	Unknown location or author
23514	check_violation	400	Rating outside 1–5; authorless review

Mapping SQLSTATE codes to HTTP status codes means the database remains the authority on what is valid; the application layer only translates. A new constraint added later produces a correct response without new application logic.

12. Spatial Data with PostGIS

12.1 Storage

PostGIS is an extension that adds spatial types, operators, indexes and roughly a thousand functions to PostgreSQL. SafeWay uses a narrow slice of it.

```
geog GEOGRAPHY(POINT, 4326) NOT NULL
```

4326 is the SRID (Spatial Reference Identifier) for WGS-84, the datum used by GPS. Choosing `GEOGRAPHY` over `GEOMETRY` means distance functions return **metres on a spheroid** rather than degrees on a plane. At Vellore's latitude (12.9°N) a degree of longitude is about 108 km while a degree of latitude is about 111 km; planar arithmetic on raw degrees would introduce roughly a 3% directional error into every radius query.

12.2 Writing a point

```
INSERT INTO locations (name, category_id, address, geog)
VALUES ($1, $2, $3, ST_SetSRID(ST_MakePoint($lng, $lat), 4326)::geography)
RETURNING id;
```

*Note the argument order: `ST_MakePoint` takes **(longitude, latitude)** — X before Y — which is the reverse of the conventional "lat, lng" ordering and a frequent source of error.*

12.3 Reading a point back

```
SELECT ST_Y(geog::geometry) AS lat,
       ST_X(geog::geometry) AS lng,
       ST_AsText(geog::geometry)   AS wkt
FROM   locations WHERE id = 4;
```

```
    lat     |    lng    |            wkt
------------+-----------+----------------------------
 12.9221524 | 79.1325219| POINT(79.1325219 12.9221524)
```

12.4 The proximity predicate

```
SELECT l.id, l.name,
       ST_Distance(l.geog, ST_SetSRID(ST_MakePoint(79.1325, 12.9165),4326)::geography) AS metres
FROM   locations l
WHERE  ST_DWithin(l.geog, ST_SetSRID(ST_MakePoint(79.1325, 12.9165),4326)::geography, 12000)
ORDER  BY metres
LIMIT  3;
```

```
  id |       name        | metres
-----+-------------------+--------
  59 | Sankari Hospital  |    167
 ...
```

`ST_DWithin(a, b, d)` is preferred to `ST_Distance(a,b) <= d` because only the former is index-assisted: the planner can rewrite it into the bounding-box operator described in §8.4, whereas a comparison against a computed distance must be evaluated for every row.

13. Query Catalogue

Representative queries by SQL sub-language, each with output taken from the live database.

13.1 DDL — schema evolution

```
-- Migration 3: widen the uniqueness rule from one review per place
-- to one review per place per time bucket.
DROP INDEX "reviews_user_id_location_id_key";
CREATE UNIQUE INDEX "reviews_user_id_location_id_time_bucket_key"
    ON reviews (user_id, location_id, time_bucket);
```

13.2 DML — erasure by anonymisation

```
BEGIN;
  UPDATE reviews SET user_id = NULL WHERE user_id = $1;   -- sever the personal link
  DELETE FROM users WHERE id = $1;                        -- remove the person
COMMIT;
```

The `UPDATE` is redundant given `ON DELETE SET NULL`, and is issued anyway so the count of anonymised rows can be reported back to the user. It also makes the intent explicit at the call site rather than relying on a schema property declared elsewhere.

13.3 DQL — aggregation across the time dimension

```
SELECT c.name AS category,
       round(avg(s.score) FILTER (WHERE s.time_bucket='morning'),  2) AS morning,
       round(avg(s.score) FILTER (WHERE s.time_bucket='afternoon'),2) AS afternoon,
       round(avg(s.score) FILTER (WHERE s.time_bucket='evening'),  2) AS evening,
       round(avg(s.score) FILTER (WHERE s.time_bucket='night'),    2) AS night,
       round(avg(s.score) FILTER (WHERE s.time_bucket='afternoon')
           - avg(s.score) FILTER (WHERE s.time_bucket='night'),    2) AS day_night_gap
FROM   location_safety_score s
JOIN   locations l           ON l.id = s.location_id
JOIN   location_categories c ON c.id = l.category_id
GROUP  BY c.name
ORDER  BY day_night_gap DESC;
```

```
  category   | morning | afternoon | evening | night | day_night_gap
-------------+---------+-----------+---------+-------+---------------
 Bus stop    |    3.58 |      3.94 |    3.41 |  2.10 |          1.84
 ATM / Bank  |    3.12 |      3.34 |    2.84 |  1.66 |          1.69
 Park        |    3.87 |      3.48 |    3.12 |  2.00 |          1.48
 Market      |    3.49 |      3.73 |    3.52 |  2.53 |          1.21
 College     |    3.90 |      3.96 |    3.63 |  3.01 |          0.95
 Café        |    3.67 |      3.89 |    3.89 |  3.46 |          0.43
 Hostel      |    3.69 |      3.62 |    3.52 |  3.34 |          0.27
 Hospital    |    4.00 |      4.08 |    4.02 |  3.88 |          0.20
```

The `FILTER` clause is SQL:2003 conditional aggregation, and pivots the four bucket rows into four columns in one pass. The result is the central empirical claim of the project: bus stops lose 1.84 points between afternoon and night while hospitals lose 0.20.

13.4 DQL — which factor degrades, not just how much

```
SELECT f.label,
       round(avg(rf.score) FILTER (WHERE r.time_bucket='afternoon'),2) AS afternoon,
       round(avg(rf.score) FILTER (WHERE r.time_bucket='night'),2)     AS night,
       round(avg(rf.score) FILTER (WHERE r.time_bucket='afternoon')
           - avg(rf.score) FILTER (WHERE r.time_bucket='night'),2)     AS drop
FROM   review_factor_ratings rf
JOIN   reviews r        ON r.id = rf.review_id
JOIN   safety_factors f ON f.id = rf.factor_id
GROUP  BY f.label, f.display_order
HAVING count(*) > 100
ORDER  BY drop DESC;
```

```
        label         | afternoon | night | drop
----------------------+-----------+-------+-------
 How busy it is       |      3.97 |  2.27 |  1.70
 Lighting             |      3.99 |  2.47 |  1.53
 Free from harassment |      4.06 |  2.61 |  1.45
 Security presence    |      3.48 |  2.66 |  0.83
```

```
 Easy to leave        |      3.42 |  2.92 |  0.50
 Transport access     |      3.79 |  3.85 | -0.06
```

The finding that justifies the schema

Transport access is the *only* factor that does not degrade at night — it is marginally better. The buses still run. Every other dimension collapses. A schema storing one score per location could not express this at all; it is visible only because the many-to-many decomposition into `review_factor_ratings` preserves each factor separately. This single row is the argument for the design.

13.5 DQL — window function

```
SELECT c.name AS category, l.name AS place, s.score,
       RANK() OVER (PARTITION BY c.name ORDER BY s.score DESC) AS rank_in_category
FROM   location_safety_score s
JOIN   locations l           ON l.id = s.location_id
JOIN   location_categories c ON c.id = l.category_id
WHERE  s.time_bucket = 'night' AND s.score IS NOT NULL
ORDER  BY c.name, rank_in_category;
```

```
 category |         place         | score | rank_in_category
----------+-----------------------+-------+------------------
 Bus stop | Vellore Old Bus Stand |  2.44 |                1
 Bus stop | Kangeyanallur road    |  2.43 |                2
 Bus stop | Lakshmi theatre       |  2.38 |                3
 Bus stop | Auxilium rountanna    |  2.33 |                4
```

13.6 DQL — correlated subquery

```
SELECT l.name, c.name AS category, s.score
FROM   location_safety_score s
JOIN   locations l           ON l.id = s.location_id
JOIN   location_categories c ON c.id = l.category_id
WHERE  s.time_bucket = 'night'
  AND  s.score > (SELECT avg(s2.score)
                  FROM   location_safety_score s2
                  JOIN   locations l2 ON l2.id = s2.location_id
                  WHERE  l2.category_id = l.category_id     -- correlation
                    AND  s2.time_bucket = 'night')
ORDER  BY s.score DESC LIMIT 5;
```

```
            name             | category | score
-----------------------------+----------+-------
 Mm Clinic                   | Hospital |  4.21
 Government Pentland Hospital | Hospital |  4.17
 Dr. Santhakumari Hospital    | Hospital |  4.04
```

13.7 DQL — distribution check with `width_bucket`

```
SELECT width_bucket(score, 1, 5, 8) AS bucket, count(*)
FROM   location_safety_score WHERE score IS NOT NULL GROUP BY 1 ORDER BY 1;
```

```
 range      | count | histogram
------------+-------+-------------------------------------------
 1.00–1.50  |     1 |
 1.50–2.00  |    13 | ####
 2.00–2.50  |    15 | #####
 2.50–3.00  |    15 | #####
 3.00–3.50  |    64 | ####################
 3.50–4.00  |   127 | ########################################
 4.00–4.50  |    45 | ###############

 min: 1.44   avg: 3.46   max: 4.39   stddev: 0.63   scored rows: 280
```

Used to demonstrate that the generated data is genuinely distributed rather than clustered at a single value — a failure mode of naive synthetic data.

13.8 DQL — verifying FR6

```
WITH per AS (SELECT user_id, location_id, count(*) n FROM reviews GROUP BY 1,2)
SELECT n AS buckets_per_person_per_place, count(*) AS pairs FROM per GROUP BY 1 ORDER BY 1;
```

```
 buckets_per_person_per_place | pairs
------------------------------+-------
                            1 |   815
                            2 |   136
                            3 |    13
                            4 |     1
```

No value exceeds 4, which is the maximum the unique constraint permits. 150 person–place pairs span more than one time of day, exercising the constraint rather than merely declaring it.

14. Performance

14.1 Database volume

Relation	Rows	Total size
review_factor_ratings	6,780	496 kB
reviews	1,130	304 kB
locations	75	72 kB
users	50	48 kB
location_categories	8	48 kB
safety_factors	6	48 kB
spatial_ref_sys (PostGIS reference data)	8,500	7,144 kB

14.2 Plan for the principal query

```
GroupAggregate (actual rows=75)
  Group Key: l.id, c.name
  ->  Sort (actual rows=287)  Sort Method: quicksort  Memory: 38kB
        ->  Nested Loop (actual rows=287)
              ->  Hash Right Join (actual rows=287)   Hash Cond: (r.location_id = l.id)
                    ->  HashAggregate (actual rows=287)      -- outer aggregate of the view
                          ->  HashAggregate (actual rows=1130) -- per-review CTE
                                ->  Hash Join (actual rows=6780)
                                      ->  Seq Scan on review_factor_ratings (6780)
                                      ->  Hash -> Seq Scan on reviews (1130)
                    ->  Hash -> Seq Scan on locations (75)
                          Filter: st_dwithin(geog, '...'::geography, 12000, true)
              ->  Index Scan using location_categories_pkey (loops=287)
Planning Time: 54.454 ms      Execution Time: 37.437 ms
```

14.3 Reading the plan

- The two stacked `HashAggregate` nodes are the view's two-stage aggregation from §9.2: 6,780 ratings collapse to 1,130 review means, then to 287 `(location, bucket)` groups.
- Only **287** of a possible 300 groups exist — 13 `(location, bucket)` combinations have no reviews at all and correctly produce no row, which the `LEFT JOIN` then renders as `NULL`.
- The `Index Scan` on `location_categories_pkey` executes 287 times, once per group. This is an indexed lookup on an 8-row table, not an N+1 problem: it is inside a single SQL statement, not a per-row round trip from the application.
- Planning time (54 ms) exceeds execution time (37 ms), which is characteristic of a complex plan over a small dataset. In the running application this query is issued once per page load and measured end-to-end at **5–6 ms** after the plan is cached.

14.4 Avoiding the N+1 pattern

The naive implementation of the map endpoint would fetch 75 locations and then issue four score queries per location — 301 round trips. Instead the four buckets are pivoted into columns with `FILTER` inside one statement. Verified by instrumenting the driver's query log:

```
/nearby        -> 75 locations in 1 SQL query
/locations/:id -> 4 SQL queries  (1 base + 3 issued in parallel), constant in review count
```

15. Database Security

15.1 Defence in depth — two independent layers

The rule "only accounts registered as female may write a review" is enforced twice, in two different systems, for a specific reason: *an application bug must not become a data-integrity breach*.

Layer	Where	Mechanism and failure mode covered
1	Application middleware	Loads the user row from the database on every request and checks `gender`. Produces a clean HTTP 403. Covers the normal path.

Layer	Where	Mechanism and failure mode covered
2	`BEFORE INSERT` trigger	Raises `SQLSTATE 42501`. Covers a bug in layer 1, a forgotten middleware on a new route, a migration script, an admin console, or any direct SQL connection.

15.2 Verification by deliberate bypass

Layer 2 is verified by a test that **opens its own PostgreSQL connection**, bypassing the application entirely, and attempts the forbidden insert:

```
const pg = new Client({ connectionString: process.env.DATABASE_URL });

await expect(
  pg.query(`INSERT INTO reviews (user_id, location_id, visited_at, time_bucket)
            VALUES ($1, $2, $3, 'morning')`, [maleUserId, locationId, visitedAt])
).rejects.toMatchObject({
  code: '42501',
  message: expect.stringContaining('female'),
});
```

```
✓ blocks a male author inserted directly via pg, bypassing the API entirely
✓ blocks an authorless review
✓ allows a female author and derives time_bucket from visited_at in IST
✓ derives every bucket boundary correctly            (all 8 edges)
✓ blocks a duplicate bucket at the database level too, bypassing the API
```

A constraint that has never been observed to reject anything is an assumption, not a guarantee. This test converts it into a guarantee.

15.3 SQL injection

No query anywhere in the system is assembled by string concatenation of user input. Two mechanisms:

- ORM-generated queries are parameterised by construction.
- Hand-written SQL uses tagged-template parameterisation. In

  ```
  await prisma.$queryRaw`SELECT … WHERE l.id = ${id}`
  ```

  the interpolation does **not** splice text into the query; the driver emits `WHERE l.id = $1` and transmits `id` separately in the Extended Query protocol. The value is never parsed as SQL.

The unsafe escape hatch (`$queryRawUnsafe`) is used exactly once, for a `TRUNCATE` statement in the seed script that contains no user input at all.

15.4 Credential storage

Passwords are stored as bcrypt digests at cost factor 10; the plaintext is never persisted or logged. Authentication tokens carry **only a subject identifier** — gender is deliberately *not* a token claim. The authorisation check re-reads gender from `users` on every request, so a forged or stale claim confers nothing. This was verified by signing a token with the genuine secret, for a real male account, carrying `{"gender":"female"}`; the request still returned 403.

15.5 Attribute-level disclosure control

Gender is a sensitive attribute. It is never projected by any unauthenticated query. This is enforced by an automated test that **enumerates the routes registered on the public router** — rather than checking a hand-maintained list — requests each one, and asserts no response body matches `/tier|gender|verif/i`. Injecting a deliberate leak causes the test to fail and name the offending URL, confirming the test is capable of failing.

Review text is likewise projected without its author: the detail query selects `body, time_bucket` and never joins to `users`. Only aggregates and unattributed text reach an unauthenticated client.

16. Data Privacy and the Right to Erasure

16.1 The conflict

India's Digital Personal Data Protection Act 2023 grants a data principal the right to erasure. A naive implementation deletes the user and cascades to their reviews. That is wrong for this system: the safety data other women depend on would silently degrade every time somebody closed an account, and a location could fall below the three-sample threshold and vanish from the map because of an unrelated person's privacy decision.

16.2 The schema-level resolution

Erasure is implemented as **anonymisation**, and this is possible only because of decisions taken in the original `CREATE TABLE`:

Design decision	Consequence
`reviews.user_id` is **nullable**	A review can exist with no author. Without this, anonymisation is impossible and only deletion remains.
`ON DELETE SET NULL`	Removing the user severs the link automatically rather than destroying the row.
Ratings hang off `reviews`, not `users`	The six factor ratings survive with their parent review, so `sample_size` is unaffected.
`UNIQUE` treats `NULL`s as distinct	Anonymised rows are exempt from FR6 and never collide (§7.3).

16.3 Verified behaviour

An automated test registers an account, posts a review, deletes the account, and asserts:

```
✓ the user row is gone                      (SELECT … WHERE email = … → null)
✓ the review still exists                   (review.userId === null)
✓ all six factor ratings survive            (count = 6)
✓ sample_size for that (location,bucket) is unchanged
✓ the token no longer authenticates         (GET /api/me → 401)
```

> **Why this belongs in a database report**
>
> Erasure looks like an application feature but is decided entirely by the schema. Once `user_id` is declared `NOT NULL` with `ON DELETE CASCADE`, no amount of application code can implement

anonymisation without a data migration. This is a clear case where a constraint chosen on day one determines whether a legal obligation can be met later.

16.4 Non-personal design constraints

- **Aggregates only.** The three-sample minimum is also a defamation control: no statement about a business is published on the authority of one or two visitors.
- **Per-venue colouring only.** The system deliberately has no area-level or neighbourhood-level aggregate, and the schema has no locality entity to compute one from. An interface that reddens whole districts becomes a redlining instrument; the omission is a design decision, not an oversight.

17. Testing the Database

Nineteen automated tests run against a live PostgreSQL instance — not a mock — so constraints, triggers and the view are exercised as they actually behave.

Suite	Tests	What it establishes
`write-gate.test.ts`	8	Trigger rejects non-female and authorless inserts via a raw connection; all 8 bucket boundaries derive correctly; four buckets accepted per person per place and a fifth rejected; forged token claim ignored.
`read-api.test.ts`	7	All four buckets returned in one response; scores serialise as JSON numbers not `Decimal` / `BigInt` ; no score below three samples; breakdown suppressed for under-sampled buckets.
`privacy.test.ts`	3	Route enumeration matches expectation; no public response mentions gender; review text carries no author.
`erasure.test.ts`	1	Anonymisation preserves the review, its ratings and the sample size.

17.1 A concurrency defect found by the suite

The erasure test initially failed intermittently with `expected 7 to be 6`. The cause was not a product defect: the four test files ran in parallel against one shared database, and a second file inserted a review into the same `(location, bucket)` between the two measurements — a textbook **phantom read** across transactions. The fix was to serialise test-file execution, which is the correct treatment for suites sharing one schema. It is recorded here because it is a genuine isolation-level phenomenon observed in practice rather than in theory.

18. Schema Evolution

Schema changes are applied as ordered, immutable SQL migration files with a ledger table (`_prisma_migrations`) recording which have been applied and a checksum of each.

Migration	Contents
`20260906065715_init`	PostGIS extension, 2 enums, 6 tables, GiST index, `CHECK` constraint, the score view
`20260906073240_review_gate`	`safeway_review_gate()` and its `BEFORE INSERT` trigger

Migration	Contents
`20260906132007_review_per_bucket`	Drop `UNIQUE(user_id, location_id)`; create `UNIQUE(user_id, location_id, time_bucket)`

The full sequence was verified to apply against an empty database (`docker compose down -v` followed by a fresh migrate and seed), which distinguishes a schema that only works incrementally from one that is genuinely reproducible.

18.1 Objects the ORM cannot express

Three categories of database object have no representation in the ORM schema language and are written as hand-authored SQL inside migrations: the `CREATE EXTENSION` statement, the `CHECK` constraint, and the trigger, function and view. The ORM's schema-diff tool was run afterwards and reported `-- This is an empty migration.`, confirming that these hand-written objects are not seen as drift and will not be reverted by a future migration — a common and destructive failure mode when mixing an ORM with procedural SQL.

Part II — Application and Operation

This part is deliberately briefer. It records how the database is reached and used; the design substance is in Part I.

19. System Architecture

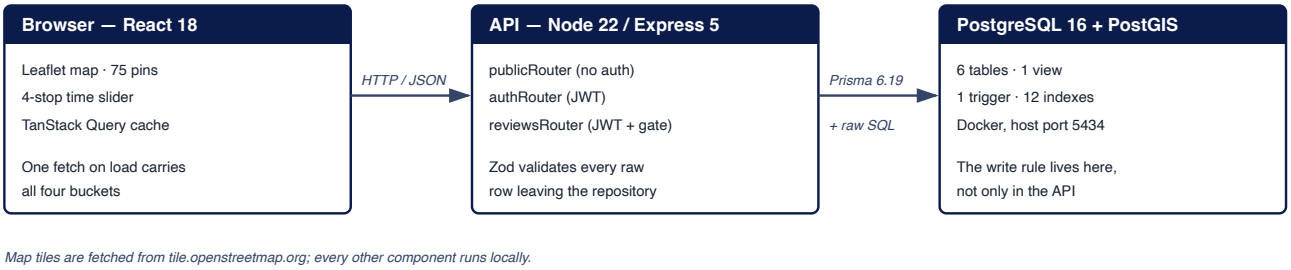


Figure 2 — Three-tier architecture. All tiers run on one machine.

19.1 Technology and versions

Layer	Technology	Note
Database	PostgreSQL 16, PostGIS 3.4	Docker image `imresamu/postgis:16-3.4`, native arm64
Data access	Prisma 6.19.3	ORM for CRUD; raw parameterised SQL for spatial and aggregate work
Runtime / API	Node.js 22.19, Express 5.2	TypeScript in strict mode
Validation	Zod 3.25	Validates both request bodies and raw-SQL result rows
Auth	jsonwebtoken 9, bcryptjs 3	Bearer tokens; bcrypt cost 10
Frontend	React 18.3, Vite 6.4, Leaflet 1.9, Tailwind 3.4	Desktop only, 1366×768 floor
Testing	Vitest 3.2, node-postgres 8.23	`pg` used to bypass the ORM in gate tests

19.2 A notable ORM boundary

Prisma cannot model the PostGIS `GEOGRAPHY` type, so `geog` is declared `Unsupported()`. The ORM responds by omitting `create`, `createMany` and `upsert` from the generated `Location` client entirely, making `prisma.location.create()` a **compile-time error**. All location writes are therefore forced through one raw-SQL repository function. The constraint turned out to be beneficial: the column is also invisible to ORM reads, so it cannot leak into an API response by accident.

20. API Surface

Method	Path	Authorisation	Database interaction
GET	`/api/locations/nearby`	None	1 spatial + aggregate query
GET	`/api/locations/:id`	None	4 queries (1 + 3 parallel)
GET	`/api/factors`	None	Reference select
POST	`/api/auth/register`	None	Insert into `users`
POST	`/api/auth/login`	None	Select by unique `email`
GET	`/api/me`	Bearer	Select by primary key
DELETE	`/api/me`	Bearer	Transaction: update then delete
POST	`/api/reviews`	Bearer + female	Transaction across two tables

21. Interface and Dataset

21.1 The time slider

The single interaction the system is built around. Because the `/nearby` response already contains all four buckets for all 75 locations, moving the slider triggers **zero** network requests and no map reload — verified by inspecting the browser network log after repeated slider movement. Measured recolouring across the same 75 pins:

Bucket	Safer	Good	Mixed	Caution	Low	No data
Afternoon	23	39	8	0	0	5
Night	3	17	15	15	20	5

The *No data* column is 5 in both, because the minimum-sample rule of §9.3 is a property of the data, not of the selected time.

21.2 Dataset provenance

The 75 locations are **real places in Vellore**, retrieved from the OpenStreetMap Overpass API and cached to disk so the seed is reproducible without network access. Users and reviews are synthetic, generated from a hidden per-category profile with per-bucket modifiers and Gaussian noise, from a fixed RNG seed — re-seeding produces a byte-identical database. Two places mapped only in Tamil script were translated; the loader drops any name it cannot render in Latin script rather than displaying it unreadably.

Category	Places	Category	Places
Bus stop	16	Market	8
Café	12	Park	7
Hospital	12	ATM / Bank	7
College	8	Hostel	5

22. Limitations

Limitation	Assessment
Gender is self-declared	The most significant limitation, and stated plainly. The database enforces the *consequence* of the declaration correctly and completely; establishing its truth requires identity verification that is outside the scope of a course project. The enforcement layer is the part that is expensive to retrofit, and it is complete.
Scoring is an unweighted mean	A location with 3 reviews and one with 30 are displayed identically. Bayesian shrinkage toward the global mean would fix this and is a one-expression change because the score is a view (§9.4). Not implemented; the minimum-sample rule already covers the failure it prevents.
Reviews are synthetic	Structure is realistic and deliberately non-uniform, but it is generated data. No conclusion about real Vellore safety should be drawn from it.
Single-node, no partitioning	Appropriate at 6,780 rows. Beyond roughly 10^7 ratings, `reviews` would be partitioned by `location_id` and the view materialised.
No moderation or incident reporting	Cut deliberately to keep the demonstrated features complete rather than numerous.

23. Future Work

1. **Materialise the score view** with an `AFTER INSERT` statement-level trigger using transition tables, once read latency exceeds the write-amplification cost.
2. **Bayesian shrinkage:** replace `AVG` with `(C·m + Σr_i) / (C + n)` in the view definition.
3. **Partition** `reviews` by `location_id` range or by `time_bucket` list at 10^7+ rows.
4. **Row-level security** (`CREATE POLICY`) to move the disclosure control of §15.5 from application projection into the database itself.
5. **Temporal versioning** of scores — a slowly-changing dimension recording how a location's score moved over months, enabling trend queries the current schema cannot answer.
6. **Verification tiers** replacing the gender boolean with a trust weight per author, turning the write gate into a weighted aggregate.

24. Conclusion

SafeWay demonstrates that the interesting decisions in a data-driven system are usually schema decisions. Three in particular determined what the finished system could do:

- Decomposing the many-to-many relationship between reviews and safety factors is what makes the result in §13.4 — transport access holding steady at night while every other factor collapses — expressible at all. A single-score schema cannot represent it.
- Declaring `reviews.user_id` nullable with `ON DELETE SET NULL` on day one is what makes lawful erasure possible without destroying aggregate data. The alternative cannot be recovered by application code.

- Placing the write rule in a `BEFORE INSERT` trigger, and the minimum-sample rule in a view, means neither can be bypassed by any consumer — a property verified by tests that deliberately bypass the application.

The system holds 6 relations, 1 view, 12 indexes, 17 constraints and 1 trigger over 8,049 rows, and satisfies all seven objectives set out in §1.3.

25. Appendix — Running the System

```
# 1 · start PostgreSQL with PostGIS (host port 5434)
docker compose up -d

# 2 · apply the three migrations, then seed
cd server && npm install && npm run db:deploy && npm run db:seed

# 3 · start the API (port 8787)
npm run dev

# 4 · start the web client (port 5174) in a second terminal
cd web && npm install && npm run dev

# tests
cd server && npm test          # 19 tests against the live database

# open a SQL session
docker exec -it safeway-dbms-db psql -U safeway -d safeway
```

Ports 5434 / 8787 / 5174 were chosen to avoid services already running on the development machine.

25.1 Verification commands used in this report

```
\dt                                      -- list relations
\d+ reviews                              -- inspect one relation
SELECT postgis_version();                -- confirm the spatial extension
SELECT * FROM pg_constraint WHERE conrelid = 'reviews'::regclass;
SELECT pg_get_viewdef('location_safety_score'::regclass, true);
SELECT pg_get_functiondef(oid) FROM pg_proc WHERE proname = 'safeway_review_gate';
EXPLAIN (ANALYZE, BUFFERS) SELECT …;     -- plans in §8.3 and §14.2
```

26. References

1. Elmasri, R. and Navathe, S. B. *Fundamentals of Database Systems*, 7th ed. Pearson, 2016. (ER modelling, ER-to-relational mapping, normal forms.)
2. Silberschatz, A., Korth, H. F. and Sudarshan, S. *Database System Concepts*, 7th ed. McGraw-Hill, 2019. (Transactions, isolation levels, indexing.)
3. Codd, E. F. "A Relational Model of Data for Large Shared Data Banks." *Communications of the ACM* 13(6), 1970.
4. PostgreSQL Global Development Group. *PostgreSQL 16 Documentation* — chapters on triggers, MVCC, index types and `information_schema`.
5. PostGIS Project. *PostGIS 3.4 Manual* — geography type, `ST_DWithin`, GiST index behaviour.
6. OpenStreetMap contributors / Overpass API. Location data for Vellore, retrieved 2026, licensed under ODbL.

7. Ministry of Electronics and Information Technology, Government of India. *Digital Personal Data Protection Act*, 2023 — right to erasure.